

A Full BLS12-381 Miller Loop Just Ran as a Bitcoin Script Predicate on Mainnet

What happens when you stop treating Bitcoin Script like a payment language and start treating it like a mathematical machine?

Over the past several weeks, I have been exploring a simple but surprisingly deep question:

How much cryptography can actually be expressed directly inside Bitcoin Script if we build upward from its primitive opcodes?

Not simulated off-chain.

Not merely committed to the blockchain.

Not verified by a separate smart-contract VM.

Actually expressed as a Bitcoin locking condition and evaluated when a transaction is spent.

The latest result pushed that experiment much further than I originally expected.

I have now deployed and spent a transaction on BSV mainnet whose locking script implements the **complete 63-round Miller loop used in a BLS12-381 pairing**.

The deployment transaction was:

```
10c52d6dfb2831ecff79fe40e827695187d1e8453af845e5ab31f17a84684d20
```

The spending transaction was:

```
f90cc1e3d60eecfb4f1a849dc4798db601b3085ca07a4155c4ae2d0890170d40
```

The locking script is **333,676 bytes**.

Inside that script are:

- 63 tangent operations
- 5 chord/addition operations
- 68 sparse line products
- 63 Fp12 squarings
- 68 witnessed Fp2 inversions
- 136 spender-supplied field values

- bounds enforcement ensuring every supplied value lies in $[0, p)$
- verification of the intermediate arithmetic required by the Miller loop

And the network accepted the spend.

That matters because this is no longer a demonstration that Bitcoin Script can perform a few large-number operations.

It is evidence that a sophisticated piece of modern pairing-based cryptography can be compiled into Bitcoin's stack-based predicate system and actually executed as part of a spend.

Why BLS12-381 Matters

BLS12-381 is one of the most important pairing-friendly elliptic curves used in modern cryptography.

It underlies or supports constructions involving:

- BLS signatures
- aggregate signatures
- threshold cryptography
- polynomial commitments
- KZG commitments
- zero-knowledge proof systems
- SNARK verification
- distributed cryptographic protocols

The important feature is a mathematical operation called a **bilinear pairing**:

$$e : G_1 \times G_2 \rightarrow G_T$$

A pairing has the property:

$$e(aP, bQ) = e(P, Q)^{ab}$$

That property creates cryptographic capabilities ordinary elliptic-curve signature systems do not provide by themselves.

At a high level, computing a BLS12-381 pairing can be separated into two major stages:

$$(P, Q) \rightarrow \text{Miller Loop} \rightarrow f \rightarrow \text{Final Exponentiation} \rightarrow e(P, Q)$$

The work now running on mainnet implements the complete first major stage.

What the Miller Loop Actually Does

It is easy to hear "63 rounds" and imagine a simple loop.

It is not.

Each iteration manipulates elliptic-curve geometry and extension-field arithmetic, accumulating line evaluations inside the twelve-degree extension field used by BLS12-381.

Conceptually, the loop repeatedly performs operations of the form:

$$f \leftarrow f^2 \cdot \ell_{T,T}(Q)$$

and, for selected bits:

$$f \leftarrow f \cdot \ell_{T,P}(Q)$$

where the line functions arise from elliptic-curve tangent or addition operations.

To express this in Bitcoin Script means reducing all of that structure down to primitives such as:

- integer multiplication
- addition
- subtraction
- modular reduction
- stack duplication
- stack indexing
- stack rotation
- conditional verification
- equality checks

There is no native opcode that says:

"Perform BLS12-381 pairing arithmetic."

There is no native Fp12 type.

There is no pairing opcode.

There is no inversion opcode designed for this curve.

The entire construction has to emerge from much simpler pieces.

That is the part I find most interesting.

Bitcoin Script as a Cryptographic Assembly Language

This project has increasingly changed the way I think about Script.

At first glance, Script appears tiny.

You see instructions such as:

OP_ADD

OP_MUL

OP_MOD

OP_PICK

OP_ROLL

OP_VERIFY

And it is easy to conclude that the language itself is mathematically primitive.

But primitive instructions do not necessarily imply primitive programs.

A CPU instruction set is also composed of very small operations.

What matters is what can be built through composition.

The experiment is therefore becoming less about individual opcodes and more about this question:

Can complex cryptographic predicates be systematically synthesized from simple Bitcoin Script primitives?

The Miller-loop result suggests the answer is much more interesting than "Script is just for simple payment conditions."

The Crucial Trick: Witnessed Computation

One of the most important architectural ideas in this work is that Bitcoin does not necessarily need to calculate every expensive value from scratch.

For example, field inversion is costly.

Suppose we want:

$$z = a^{-1} \pmod{p}$$

A naïve implementation would require Script itself to compute the inverse.

Instead, the spender can provide a proposed inverse:

$$z$$

and Bitcoin Script verifies:

$$a \cdot z \equiv 1 \pmod{p}$$

This changes the architecture dramatically.

Instead of:

Bitcoin computes everything

we can use:

The spender computes expensive intermediate values, and Bitcoin proves that those values are correct.

That pattern appears throughout the current Miller-loop implementation.

There are **68 witnessed Fp2 inversions**.

The unlocking transaction provides intermediate values.

The locking script does not trust them.

It constrains them.

It verifies them.

This is a powerful general pattern because verification can be far cheaper than reconstruction.

136 Numbers Chosen by the Spender — None Trusted

The current script accepts **136 numbers selected by the spender**.

But every one is constrained to the BLS12-381 base field:

$$0 \leq x < p$$

and every mathematically meaningful relationship between those values is checked.

That gives us something structurally similar to a verified computation model:

```
Prover / spender
  |
  | expensive intermediate values
  v
Bitcoin Script
  |
  | range checks
  | modular equations
  | curve relations
  | field identities
  v
TRUE
```

The values are not accepted because the spender supplied them.

They are accepted because the predicate proves their consistency.

That distinction opens up a much broader design space.

What Has Been Proven — and What Has Not

It is important to be precise.

I am **not** claiming that a complete BLS12-381 pairing currently executes in one relayable mainnet script.

The current measurements are:

Component	Script Size	Current Status
Miller loop	333,676 bytes	Mainnet
Final exponentiation	592,008 bytes	Interpreter
Full pairing <code>e(P,Q)</code>	935,388 bytes	Interpreter

The complete pairing currently exceeds the approximately 500 KB relay-policy boundary encountered by the implementation.

So the accurate statement is:

The complete 63-round Miller-loop stage of BLS12-381 pairing computation has executed successfully as a Bitcoin Script predicate on mainnet.

That is already a substantial result.

The remaining problem is not whether the arithmetic can be expressed.

It is how to package and optimize the complete computation efficiently enough for practical deployment.

That is a very different research problem.

The Final Exponentiation Is Now the Main Target

The size profile is revealing.

The Miller loop currently occupies:

333,676 bytes

while final exponentiation requires:

592,008 bytes

The full pairing totals:

935,388 bytes

So roughly 63% of the present full-pairing implementation is the final exponentiation.

That gives the optimization work a very clear target.

Possibilities include:

- better addition chains
- exploiting Frobenius operations more aggressively
- specialized cyclotomic squaring
- improved Fp12 multiplication strategies
- more witness-assisted subcomputations
- compiler-level stack optimization
- constant folding
- common-subexpression elimination
- transaction-level decomposition

The interesting thing is that these are now engineering questions.

The foundational expressiveness question has already moved significantly.

Why the First Mainnet Broadcast Failed

There was also an unexpected lesson hiding in the deployment process.

The first broadcast failed.

The problem was not the cryptography.

It was the wallet.

Large scripts exposed assumptions that had never mattered when transactions were only a few hundred bytes.

Two real bugs appeared.

Bug #1: Dust First

The wallet's `spendable()` function returned outputs in indexer order.

That meant the transaction builder could consume tiny unconfirmed outputs before using a large confirmed or fresh output.

Those dust outputs happened to sit at the end of chains created by earlier deployment transactions.

A fresh 50,000-satoshi UTXO could fund the roughly 334 KB transaction by itself.

But when the builder selected several satoshis of dust first, the resulting transaction inherited a long chain of unconfirmed ancestors.

The node rejected it with:

```
too-long-mempool-chain
```

The fix was simple but important:

within each confirmation state, select larger UTXOs first.

The pairing script had become large enough to expose an assumption in ordinary wallet coin selection.

Bug #2: A 5,000-Satoshi Fee Buffer Is Not a Fee Model

The deployment builder also stopped looking for inputs when it had:

```
required output satoshis + 5000
```

That worked for small covenants.

It did not make sense for a locking script whose fee alone was around **33,389 satoshis**.

The code technically did not underfund the transaction.

Instead, it simply kept collecting wallet inputs.

And that behavior amplified the first bug by sweeping more dust.

The builder now calculates the funding requirement from the actual script being deployed.

If funds are insufficient, it reports how much is missing and why.

Again, this was not merely a wallet fix.

It was evidence of a transition from experimental Script examples into workloads large enough to stress surrounding infrastructure.

Why I Kept the 48-Round Version

Earlier in the work I had a 48-round Miller-loop prefix.

When the complete 63-round implementation worked, I deliberately did not replace the older target.

Both remain independently testable.

The function takes the round count as a parameter:

```
pairing.miller(n)
```

That matters because a truncated Miller loop becomes a real reproducible object rather than a temporary demonstration.

The progression now looks like this:

```
finite-field arithmetic
  |
  v
Fp2 arithmetic
  |
  v
Fp6 / Fp12 arithmetic
  |
  v
elliptic-curve line functions
  |
  v
truncated Miller loop
  |
  v
48-round Miller loop
  |
  v
63-round Miller loop
  |
  v
FULL MILLER LOOP
  |
  v
final exponentiation
  |
  v
complete pairing
  |
  v
BLS verification
```

Each stage remains independently checkable.

That provides evidence of composition rather than only showing the latest artifact.

The Bigger Goal Is Not a 333 KB Script

The most important realization from this work is that manually constructing increasingly enormous Script programs is not the final objective.

The interesting project is a compiler.

Something closer to:

```
high-level cryptographic specification
      |
      v
mathematical IR
      |
      v
witness planner
      |
      v
arithmetic compiler
      |
      v
stack optimizer
      |
      v
Bitcoin Script
      |
      v
network-enforced predicate
```

A developer should not need to manually reason about thousands of `OP_PICK` and `OP_ROLL` instructions.

They should write something closer to:

```
verifyPairing(P, Q, expected)
```

or:

```
requireValidBLSSignature(
    publicKey,
```

```
message,  
signature  
)
```

and allow a compiler to lower that requirement into the necessary Script primitives.

That is where this work appears to be heading.

From Pairings to BLS Signatures

A BLS signature verification is fundamentally a pairing equation.

Depending on convention, verification can be represented approximately as:

$$e(\sigma, G_2) = e(H(m), PK)$$

where:

- m is the message
- $H(m)$ is the message mapped to an elliptic-curve point
- σ is the signature
- PK is the public key

Once pairing verification becomes practical, a Bitcoin output could potentially enforce:

This UTXO may only be spent if the spender supplies a valid BLS signature.

Bitcoin would not merely contain a BLS signature.

Bitcoin Script would verify the foreign cryptosystem itself.

That distinction is important.

Aggregate Signatures Become Particularly Interesting

One of the major advantages of BLS is signature aggregation.

Given signatures:

$$\sigma_1, \sigma_2, \dots, \sigma_n$$

they can be combined into an aggregate signature.

This makes BLS attractive for systems involving:

- large validator groups
- organizational approvals
- threshold authorization
- decentralized identity
- distributed signing systems

A successful pairing verifier would therefore give Bitcoin Script a path toward enforcing cryptographic authorization models that it was never explicitly given native opcodes for.

Again, the larger principle is:

If the mathematics can be reduced to primitives Script already understands, native opcode support may not be necessary.

And Then Things Get More Interesting: KZG and SNARKs

Pairings are not only about BLS signatures.

They sit underneath some of the most powerful proof systems in modern cryptography.

Consider a polynomial commitment.

A system can commit to an enormous polynomial:

$$f(x)$$

and later prove that:

$$f(z) = y$$

without revealing or recomputing the entire polynomial.

Pairing-based systems such as KZG make those proofs compact.

The verification operation ultimately reduces to elliptic-curve and pairing equations.

That creates an intriguing possibility:

```
large computation
  |
  v
succinct proof
  |
  v
pairing verification
  |
  v
Bitcoin Script
  |
  v
TRUE / FALSE
```

This is much more powerful than trying to reproduce every large computation directly inside Script.

Instead, Bitcoin could verify a compact proof that the computation occurred correctly.

Toward SNARK Verification in Bitcoin Script

Groth16-style SNARK verification also relies heavily on pairing equations.

A SNARK verifier does not execute the original program.

It verifies a proof.

Conceptually:

```
arbitrary computation
  |
  v
proof
  |
  v
few elliptic-curve operations
  |
  v
pairing equation
  |
  v
TRUE
```

If pairing equations can be compiled into Bitcoin Script efficiently enough, then the question becomes:

Can Bitcoin Script verify succinct proofs of computations vastly larger than the Script itself?

That is a much larger idea than implementing BLS.

It moves Bitcoin Script toward becoming a universal verification layer.

Multi-Transaction Cryptographic Computation

The 500 KB policy constraint also raises another interesting possibility.

A complete computation does not necessarily need to exist inside one transaction.

Bitcoin already provides a state-transition mechanism:

UTXOs.

Instead of:

```
one UTX0
|
935 KB computation
|
result
```

we can investigate something closer to:

```
UTX0 A
|
Miller loop
|
v
verified intermediate state
|
v
UTX0 B
|
final exponentiation stage
|
v
UTX0 C
|
final verification
```

|
v
result

The challenge is ensuring that each output cryptographically commits to the verified state of the previous stage.

But if done correctly, Bitcoin transactions themselves become the stages of a cryptographic computation.

That creates an entirely different programming model:

stateful, transaction-linked, cryptographically enforced computation.

This Is Really About Expressiveness

The broader lesson from this work is not that everyone should start deploying 333 KB pairing scripts.

It is about understanding what a primitive scripting language can express.

Bitcoin Script gives us operations.

Mathematics gives us identities.

Witnesses allow expensive operations to be converted into cheaper verification problems.

A compiler can connect the three.

The resulting model looks like this:

mathematical statement

becomes:

witness + constraints

which becomes:

Bitcoin Script

which becomes:

a spendable or unspendable UTXO

The blockchain is no longer simply recording that some computation happened.

The validity of the computation can become part of the condition for transferring value.

Where This Work Stands Today

The current state is:

Complete BLS12-381 Miller loop

333,676-byte Bitcoin locking script.

63 Miller rounds.

Mainnet deployment.

Mainnet spend.

Final exponentiation

592,008-byte implementation.

Passing in the interpreter.

Complete pairing

935,388-byte implementation.

Passing in the interpreter.

Currently above the relay-policy size boundary being targeted.

The next technical challenge is therefore straightforward to state:

Reduce, restructure, or distribute final exponentiation sufficiently to make complete pairing verification practical on-chain.

After that comes the application layer:

- BLS verification
- aggregate BLS
- threshold signatures
- polynomial commitments
- pairing-based proof verification
- potentially SNARK verification

What I Think This Demonstrates

I started this line of research with a fairly simple idea:

Bitcoin Script might be much more expressive than people assume if we stop asking what individual opcodes were "intended" to do and instead ask what mathematical constructions can be composed from them.

The full Miller-loop result provides a concrete example.

A pairing-friendly elliptic curve was certainly not encoded into Bitcoin as a native primitive.

Yet enough basic mathematical machinery exists to construct its core pairing algorithm anyway.

That leads me to a broader thesis:

Bitcoin Script's apparent simplicity should not be confused with computational or mathematical simplicity.

Complex cryptographic verifiers can potentially be synthesized from primitive operations.

Expensive calculations can be moved into witness generation.

Bitcoin can verify the relationships between those witnesses.

And a compiler can eventually hide the enormous amount of low-level stack manipulation required to make it happen.

The most interesting outcome of this work may therefore not be a BLS implementation at all.

It may be a new way of thinking about Bitcoin programming:

Write the mathematical predicate you want enforced.

Let the compiler figure out how Bitcoin can prove it.

That is the direction I am exploring now.

— **Gregory Ward**

CTO / Co-Founder, SmartLedger Blockchain Solutions